Text line length adjustment

Given a set of lines of text of arbitrary length, reformat the text so that no lines of more than n characters are printed. In each output line the maximum number of words that occupy less than or n characters, should be printed and no word should extend across 2 lines. Paragraphs should also remain indented.

- tabs are not be considered(take them as single space)
- Every end of the line returns a space.

Algorithm

1. Establish the line length limit *limit* and add one to it to allow for a space.
2. Initialize word and line character counts to zero and end-of-line flag to *false*
3. While not end-of-file do

a) read and store next character

b) if character is a space then

b.1) if a new paragraph then

1.a) move to next line and reset character count for the new line.

b.2) add current word length to current line length

b.3) if current word causes line length limit to be exceeded
then
3.a) move to next line and set line length to current
word length.
b.4) write out current word and its trialing space and reinitialize character count.
b.5) turn off end-of-input-line flag
b.6) if at end-of-input-line then
6.a) set end-of-input-line flag and move to next input line.

Left and Right Justification of Text

Design and implement a procedure that will left and right justify text in a way that avoids splitting words and leaves paragraphs indented. An attempt should also be made to distribute the additional blanks as evenly as possible in the justified line.

Left and Right Justification of Text

- Fixed line length is achieved by inserting additional spaces between words.
- For any particular line the following holds
  - The line is already in correct length so no processing is needed.
  - The number of extra spaces needed to expand the current line to the required length is equal to number of spaces already present in the line.
    - It is simply adding 1 space to each existing space.
    - number of spaces to be added > existing spaces
    - number of spaces to be added < existing spaces.

Ex:

10 extra spaces to a line which has already 7 spaces.

- first add 7 spaces evenly to existing spaces
- 3 are left
  - if 1 space then add it to middle space.
  - if 2 spaces then first would be positioned one-third of the way and second, two-thirds of the way across.
  - if 3 spaces then add after 2, 4 and 6 word

Algorithm

1. Establish line to be justified, its current length and justification length.
2. Include test to see if it can be justified.
3. Initialize space count and alphabetic start of line.
4. while current character a space do
   a) shift to next character;
   b) increment alphabetic start to end to line.
   c) write out a space
5. For from alphabetic start to end of line do
   a) If current character is space then
   a.1) increment space count
   a.2) set current position in space count table to 1
6. Remove any spaces from end of line

7. Determine the extra spaces to be added from new and old line lengths.
8. While still extra spaces to add and possible to do so do
   a) compute current template increment from space count and extra spaces count
   b) if increment 0 the set to 1 since more extras than spaces
   c) if extra spaces > space count then
   c.1) determine space block using extra spaces and space
   count else
   c.1`) set space block size to 1
   d) determine starting position for template
   e) while not end-of-line and still spaces to add do
   e.1) add space block to current template position.
   e.2) move to next position in template
   e.3) decrement extra space count by space block size

9. For from start to end-of-line do

a) if next character a space then

a.1) move to next position in space count table

a.2) write out number of spaces as per space count table

else

a.1`) write out current character

10. Finish off with an end-of-line.

Keyword Searching in Text

count the number of times a particular word occurs in a given text

Algorithm

1. Establish the word and word length *wlength* of the search-word
2. Initialize the match-count *nmatches*, set preceding character and set pointer for word array i to 1
3. while not at end-of-file do

   a) while not end-of-line do

   a.1) read next character

   a.2) if current text character *chr* matches i^{th} character in word then

   2.a) extend partial match i by 1,

   2.b) if a word-pattern match then

   b.1) read next character post,

b.2) if preceding and following character not alphabetic then

2.a) update match count *nmatches*

b.3) reinitialize pointer to word array i

b.4) save following character post as preceding character

else

2.a`) save current text character as preceding character for match

2.b`) reset word array pointer i to first position

b) read past end-of-line.

4. return word-match count *nmatches*

Text Line Editing

Design and implement an algorithm that will search a line of text for a particular pattern or substring. Should the pattern be found it is to be replaced by another given pattern.

the two wrongs in this line are wrong --original line

the two rights in this line are right -- edited line

Algorithm

1. Establish the textline, the search pattern and replacement pattern and their associated lengths.
2. Set initial values for the position in the old text, the new text and the search pattern
3. While all pattern positions in the text have not been examined do
   a) if current text and pattern characters match then
      a.1) extend indices to next pattern/text character pair
   a.2) if a complete match then
      2.a) copy new pattern into current position in edited line
      2.b) move past old pattern in text
      2.c) reset pointer for search pointer.
   else

a.1) copy current text character to next position in edited text

a.2) reset search pattern pointer

a.3) move pattern to next text position

4. Copy the leftover characters in the original text line

5. Return the edited line of text

Linear Pattern Search

Design and implement a pattern searching algorithm with a performance that is linearly dependant on the length of the string or text being searched. A count should be made of the number of times the search pattern occurs in the string

Algorithm Description

- Partial-match table setup algorithm
- Linear pattern searching algorithm
- Procedure for recovering from mismatches and complete matches.

Partial-match table setup algorithm

1. Establish the search pattern
2. Set initial displacement between the pattern and itself to one.
3. Initialize the zero and first position in the partial match array to zero
4. while all positions of pattern relative to itself not considered do

a) if current pattern and displaced pattern character pairs match then

a.1) save current degree of partial match

a.2) move to next position in pattern and displaced pattern

else

a`.1) a mismatch so set partial match to zero

a`.2) reset pointer to start of displaced pattern

a`.3) move the start of the displaced pattern to the next available position

5. return the partial match table.

Linear pattern searching algorithm

1. Establish the pattern to be searched for and the string in which it is to be sought together with lengths of the pattern and the string.
2. Set initial values for start of pattern and string and zero the match count.
3. while all appropriate pattern positions in the string have not been examined do

   a) if current string and pattern characters match then

      a.1) extend indices to next pattern/string pair.

      a.2) if a complete match then

   2.a) update complete match count

   2.b) reset recovery position from the partial match table

   else

   a`.1) reset recovery position from the partial match table
4. return count of the number of complete matches of the pattern in the string.

Procedure for recovering from mismatches and complete matches.

1. Establish the partial match table, the current position in the string and position in pattern
2. if no smaller partial match then

   a) move to next position in string

   b) return to start of pattern

   else

   a`) recover from mismatch or complete match by using table to set new smaller partial match for current position in string
3. return smaller partial match and current pattern position.

SubLinear Pattern Search

Design and implement an algorithm that will efficiently search given text for a particular keyword or pattern and record the number of times the keyword or pattern is found.

Algorithm

1. Establish the word and text to be searched
2. set up the skip table
3. set keyword match count to zero
4. set character position I to keyword length
5. while current character position < textlength do
   a) get numeric value nxt of current character at position I
   b) index into skip table at position nxt.
   c) if skip value for current character > 0 then
   c.1) increase current position by skip value
   else
   c`.1) backwards-match text and word
   c`.2) if match made update match count
   c`.3) recover from mismatch
6. return match count.